

AI-Powered Sales Prediction for Retail Products

STEP-1: Problem Definition

Prediction Goal: Estimate the gross income generated per transaction.

Problem Type: Regression, because the target variable is continuous.

Primary Users: Supermarket managers and analysts who need reliable forecasts to improve inventory planning, staffing decisions, and promotional strategy.

Introduction

Retail businesses generate vast amounts of transactional data every day, yet much of it remains underused unless it is transformed into actionable insight. In supermarkets, even small changes in pricing, customer behavior, product mix, and timing can significantly affect profitability. A predictive model that estimates gross income per transaction can help decision-makers anticipate future performance and respond more effectively to shifting demand patterns.

This project aims to convert raw sales records into a practical forecasting tool that supports better business decisions, reduces waste, and improves overall profitability.

Supermarkets operate in highly competitive environments where profit margins are thin and customer demand is unpredictable. Every transaction carries valuable information about purchasing behavior from product line and payment method to time of day and customer type. Without predictive insights, managers rely on descriptive reports that only explain what happened, not what will happen.

By building a model that predicts gross income per transaction, supermarkets can:

Optimize inventory to reduce waste and avoid stockouts.

Plan staffing levels more accurately, aligning workforce with peak demand hours.

Design targeted promotions based on product line and customer type.

Improve financial forecasting, ensuring better cash flow and resource allocation.

In short, solving this problem transforms raw transactional data into actionable intelligence, enabling smarter decisions and stronger profitability.

STEP-2 Data collection

This project uses the Supermarket Sales dataset, a retail transaction dataset with details such as branch, city, customer type, product line, payment method, quantity, unit price, sales, gross income, and customer rating. The data is sourced from a public supermarket sales dataset and is stored locally in the project folder at data/raw/SuperMarket Analysis.csv.

STEP 3: Load & Inspect Dataset

```
In [60]: from pathlib import Path

candidate_paths = [
    Path.cwd() / "data" / "raw" / "SuperMarket Analysis.csv",
    Path.cwd().parent / "data" / "raw" / "SuperMarket Analysis.csv",
    Path("/content/drive/MyDrive/DEEPTech MENTORSHIP/SuperMarket Analysis.csv"),
]

extract_path = None
for path in candidate_paths:
    if path.exists():
        extract_path = path
        break

if extract_path is None:
    raise FileNotFoundError(
        "Dataset not found. Place 'SuperMarket Analysis.csv' in the project data/raw"
    )
print(f"Using dataset: {extract_path}")
```

Using dataset: c:\Users\USER\AI-Workspace\ai-powered_sales_prediction_for_retail\data\raw\SuperMarket Analysis.csv

```
In [61]: import pandas as pd

# Load the dataset from the resolved local path
if 'extract_path' in locals() and extract_path is not None:
    df = pd.read_csv(extract_path)
    print("Dataset loaded successfully.")
    print(df.head())
else:
    print("No dataset path was resolved.")
```

Dataset loaded successfully.

	Invoice ID	Branch	City	Customer type	Gender	\
0	750-67-8428	Alex	Yangon	Member	Female	
1	226-31-3081	Giza	Naypyitaw	Normal	Female	
2	631-41-3108	Alex	Yangon	Normal	Female	
3	123-19-1176	Alex	Yangon	Member	Female	
4	373-73-7910	Alex	Yangon	Member	Female	

	Product line	Unit price	Quantity	Tax 5%	Sales	Date	\
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	

	Time	Payment	cogs	gross margin percentage	gross income	\
0	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	
1	10:29:00 AM	Cash	76.40	4.761905	3.8200	
2	1:23:00 PM	Credit card	324.31	4.761905	16.2155	
3	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	
4	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	

	Rating
0	9.1
1	9.6
2	7.4
3	8.4
4	5.3

Essentially, this line tells the program exactly where to find the CSV file so it can be loaded into a pandas DataFrame using `pd.read_csv(extract_path)`.

SETP-4: Data Cleaning

```
In [62]: df.isnull().sum() # Check for missing values
```

```
Out[62]: Invoice ID      0
Branch      0
City        0
Customer type  0
Gender      0
Product line  0
Unit price  0
Quantity    0
Tax 5%      0
Sales       0
Date        0
Time        0
Payment     0
cogs        0
gross margin percentage  0
gross income  0
Rating      0
dtype: int64
```

Why Missing Values Matter in This Project

Missing values are entries that are absent or blank in the dataset. In a retail sales project, they can affect the quality of the analysis and the reliability of the predictive model. For example, if a transaction is missing its product line, payment method, or quantity, it may distort customer behavior insights and weaken the model's ability to learn meaningful patterns.

Checking for missing values is important because it helps us confirm whether the data is complete enough for analysis. In this project, the goal is to predict gross income from transaction-level features, so any missing information could reduce the accuracy of the model or introduce bias.

The code below uses `df.isnull().sum()` to count missing values in each column. This helps us quickly identify whether any fields need cleaning, imputation, or removal before training the model.

In this dataset, the check showed that there are no missing values, which means the data is already complete and does not require missing-value treatment before proceeding with preprocessing and modeling.

```
In [63]: df.duplicated().sum() # Check for duplicates
```

```
Out[63]: np.int64(0)
```

Why Duplicate Checks Matter

Duplicate rows can distort the analysis by giving extra weight to the same transaction. In this project, that could make sales patterns and model training appear stronger or more frequent than they really are.

The code below checks for repeated records so the dataset stays reliable before modeling. In this case, no duplicates were found, which means the transaction data is already clean at the row level.

```
In [64]: df.dtypes # Inspect data types
```

```
Out[64]: Invoice ID          str
Branch          str
City            str
Customer type   str
Gender          str
Product line    str
Unit price      float64
Quantity        int64
Tax 5%          float64
Sales           float64
Date            str
Time            str
Payment         str
cogs            float64
gross margin percentage float64
gross income     float64
Rating          float64
dtype: object
```

Understanding the Data Types

The dataset contains a mix of numeric and categorical variables. Numeric columns such as quantity, unit price, sales, and gross income are needed for calculations and modeling, while columns such as branch, city, and payment method are categorical and must be encoded before use.

Checking the data types helps ensure that each feature is handled correctly during preprocessing and that the model receives the right format of input.

Descriptive Statistics

```
In [65]: df[["Unit price","Quantity","Sales","gross income","Rating"]].describe() # Summary
```

```
Out[65]:
```

	Unit price	Quantity	Sales	gross income	Rating
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	55.672130	5.510000	322.966749	15.379369	6.97270
std	26.494628	2.923431	245.885335	11.708825	1.71858
min	10.080000	1.000000	10.678500	0.508500	4.00000
25%	32.875000	3.000000	124.422375	5.924875	5.50000
50%	55.230000	5.000000	253.848000	12.088000	7.00000
75%	77.935000	8.000000	471.350250	22.445250	8.50000
max	99.960000	10.000000	1042.650000	49.650000	10.00000

```
In [66]: for col in ["Branch","City","Customer type","Gender","Product line","Payment"]:
          print(f"\n{col} distribution:\n", df[col].value_counts()) # Frequency counts fo
```

Branch distribution:

```
Branch
Alex      340
Cairo     332
Giza      328
Name: count, dtype: int64
```

City distribution:

```
City
Yangon    340
Mandalay  332
Naypyitaw 328
Name: count, dtype: int64
```

Customer type distribution:

```
Customer type
Member      565
Normal      435
Name: count, dtype: int64
```

Gender distribution:

```
Gender
Female     571
Male       429
Name: count, dtype: int64
```

Product line distribution:

```
Product line
Fashion accessories    178
Food and beverages    174
Electronic accessories 170
Sports and travel      166
Home and lifestyle     160
Health and beauty      152
Name: count, dtype: int64
```

Payment distribution:

```
Payment
Ewallet      345
Cash          344
Credit card  311
Name: count, dtype: int64
```

Descriptive Statistics Summary

Descriptive statistics help us understand the overall shape of the data before modeling. They show the typical values, spread, and range of key features such as quantity, unit price, sales, gross income, and customer rating.

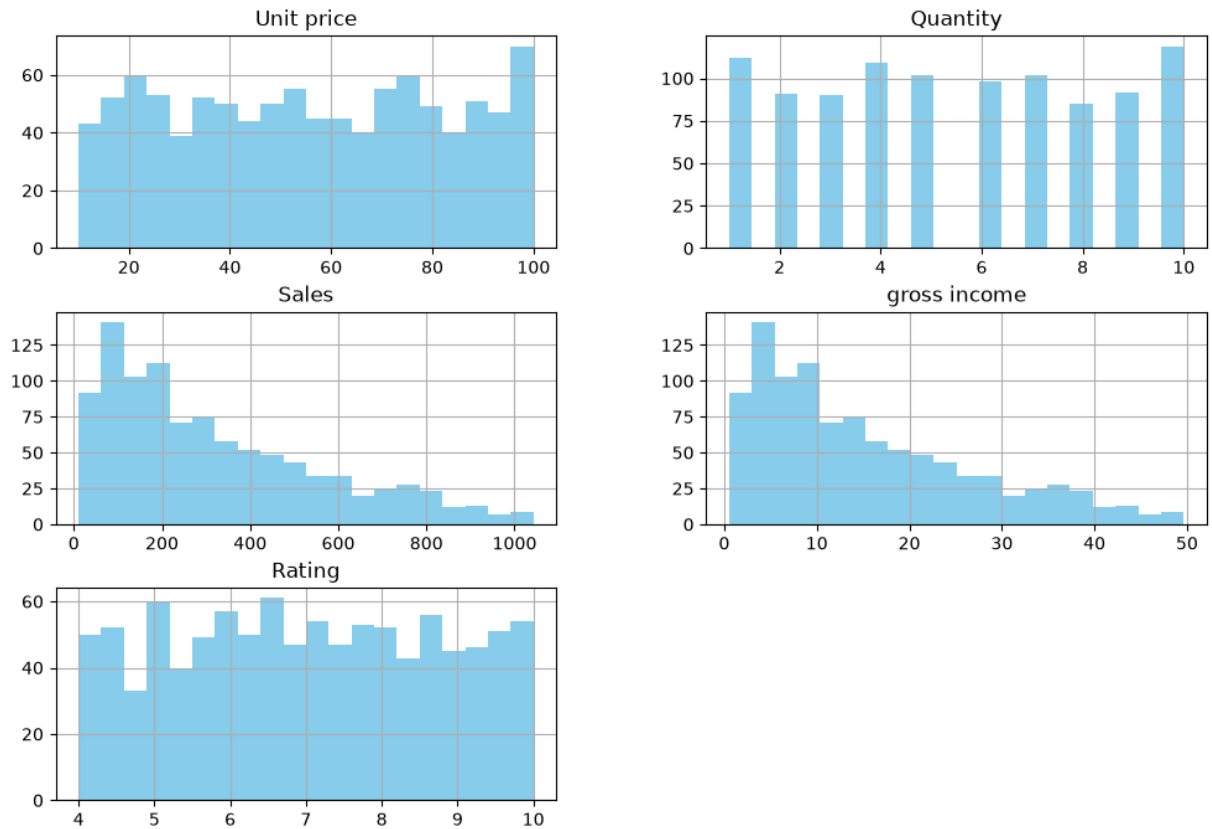
These summaries provide a quick baseline for identifying unusual patterns, comparing variables, and preparing the data for deeper analysis.

STEP-5 Exploratory Data Analysis

Distribution Analysis

```
In [67]: import matplotlib.pyplot as plt
df[["Unit price","Quantity","Sales","gross income","Rating"]].hist(bins=20, figsize=
plt.suptitle("Distribution of Continuous Variables")
plt.show() # Histograms for continuous variables
```

Distribution of Continuous Variables



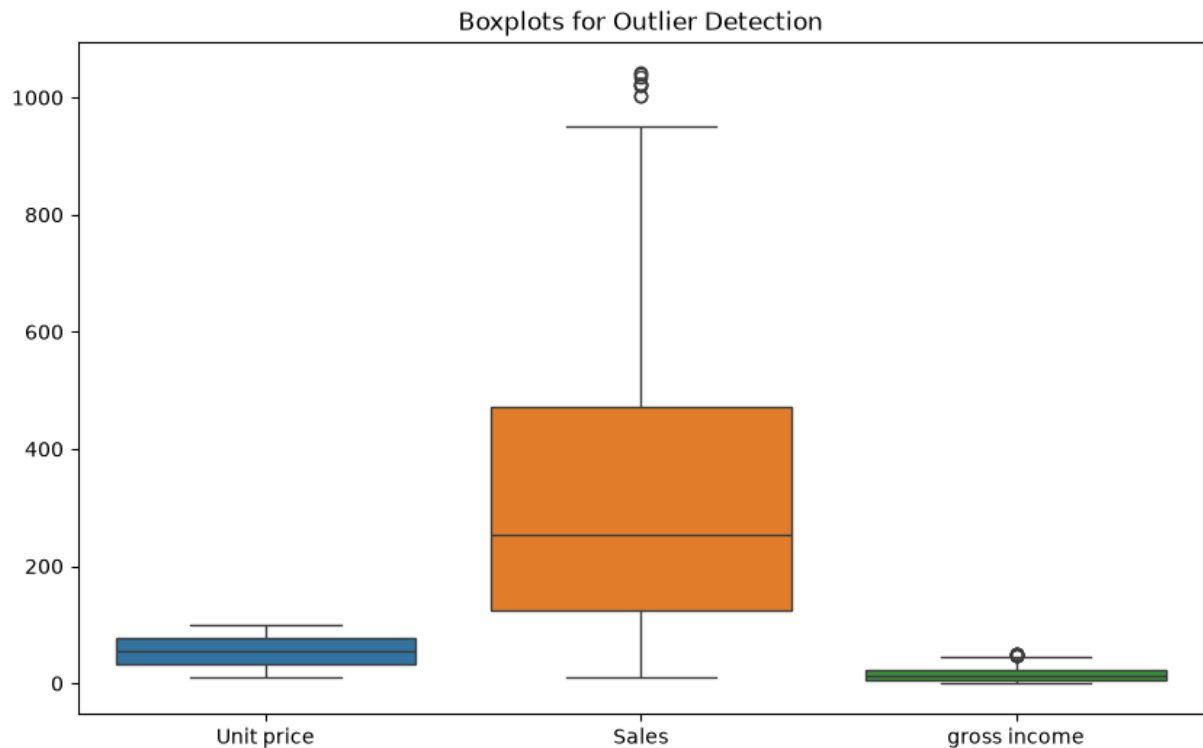
Distribution Overview

These histograms provide a visual summary of the shape of the most important numeric features in the dataset. They help us see whether values are concentrated in a narrow range, spread widely, or skewed toward a few very high or very low transactions. In a sales prediction problem, this is important because the model learns better when we understand the overall distribution of the variables it will use.

For example, a histogram with a long right tail often suggests that a small number of transactions have unusually large values. That pattern can influence the model's behavior, because regression models may be pulled strongly by these extreme cases. By inspecting the shape of the distributions early, we can decide whether further cleaning, transformation, or scaling is needed before training.

In this project, these plots also help us confirm that variables such as unit price, quantity, sales, gross income, and rating behave in a realistic way for retail transactions. They give us the first strong clue about the kind of relationships the model is likely to learn later.

```
In [68]: import seaborn as sns
# Boxplots for outlier detection
plt.figure(figsize=(10,6))
sns.boxplot(data=df[["Unit price", "Sales", "gross income"]])
plt.title("Boxplots for Outlier Detection")
plt.show() # Boxplots for outlier detection
```



Outlier Detection

Boxplots are especially useful for spotting extreme values that may distort the analysis. In this project, an outlier could represent a transaction with unusually high sales, an unusually large quantity, or a very high gross income. Such cases are important because they can pull the average upward and make the model overemphasize rare events.

The boxplot shows the spread of each variable, the typical range of values, and any points that fall far outside the normal pattern. If many extreme values appear, it may suggest that the data contains unusual transactions that need special handling. If the values are relatively stable, the data is likely suitable for modeling without major adjustment.

This step is therefore not just about cleaning the data; it is also about understanding the business context. A few very large retail transactions may be legitimate, but they should be noticed so they do not silently distort the model's learning process.

Relationship Analysis

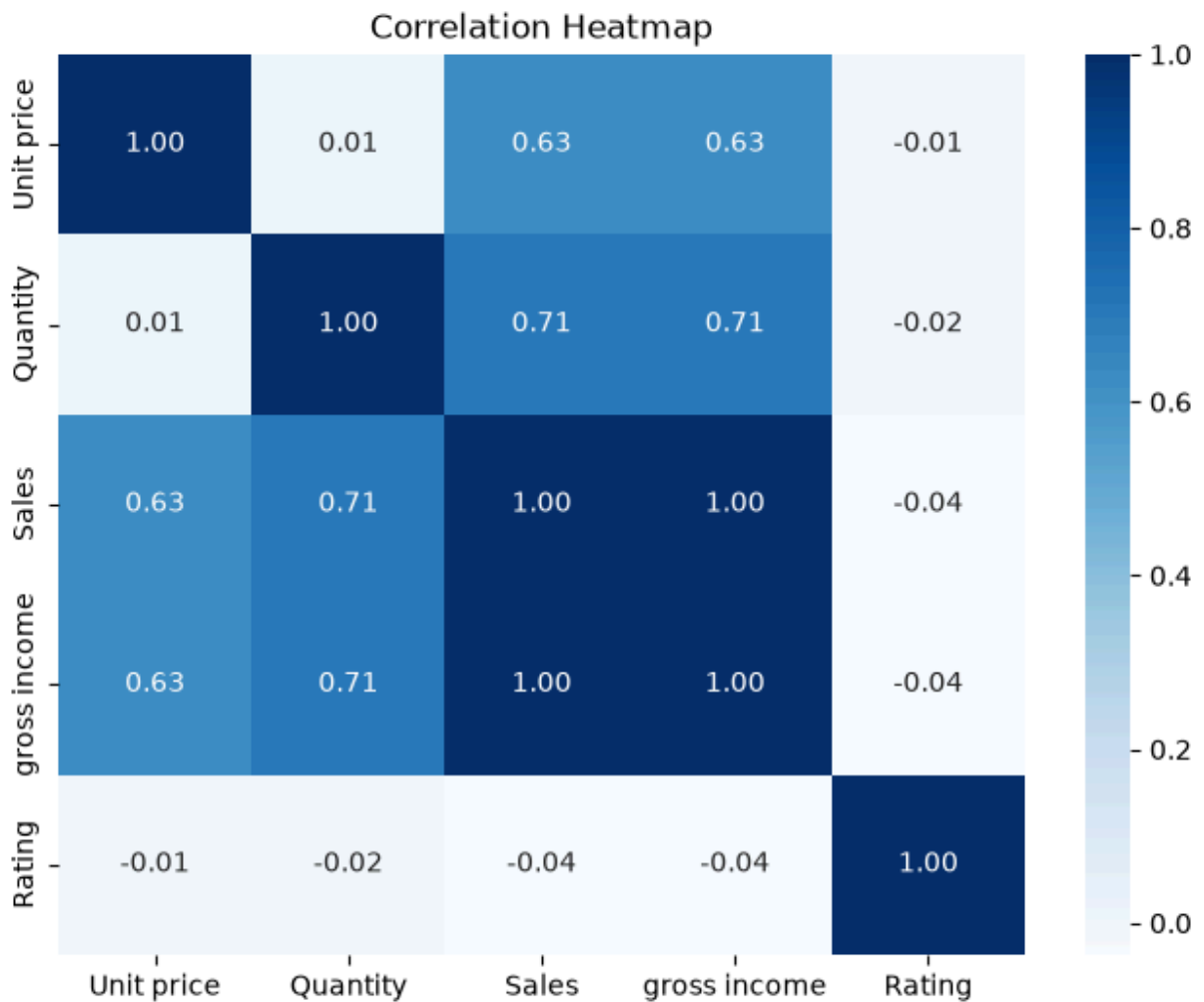
```
In [69]: # Correlation heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df[["Unit price", "Quantity", "Sales", "gross income", "Rating"]].corr(),
```



```

    annot=True, cmap="Blues", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

```



Correlation Insight

The correlation heatmap shows how strongly numeric features are related to one another. A value close to 1 or -1 indicates a strong relationship, while a value near 0 indicates little or no linear relationship. In this project, this is helpful because the target variable is gross income, and we want to understand which features are most likely to explain it.

From the heatmap, we can see whether variables such as quantity, sales, and unit price move together in a meaningful way. If gross income is strongly related to sales, that tells us the target is being driven by transaction size. This is valuable information because it confirms that the model will likely benefit from features that represent purchase volume and price.

At the same time, the heatmap also helps us think about redundancy. If two features are almost perfectly correlated, one of them may add little new information. This is important when we later refine the model and choose the most useful inputs.

```

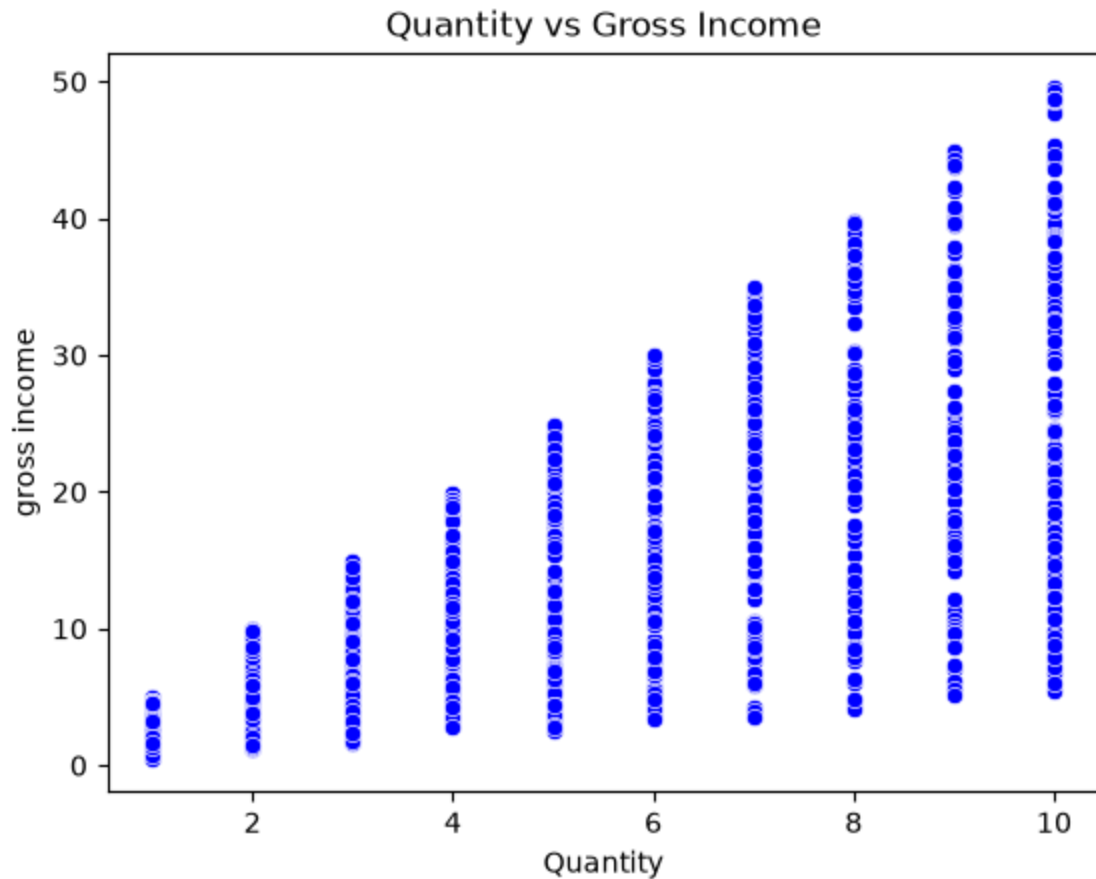
In [70]: # Scatter plots
sns.scatterplot(x="Unit price", y="gross income", data=df, color="green")

```

```
plt.title("Unit Price vs Gross Income")  
plt.show()
```



```
In [71]: sns.scatterplot(x="Quantity", y="gross income", data=df, color="blue")  
plt.title("Quantity vs Gross Income")  
plt.show()
```



The scatter plots provide a more detailed view of how two continuous variables relate to each other. Each point represents one transaction, and its position shows the values of the two features being compared. This makes it easier to see whether the relationship is strong, weak, or missing altogether.

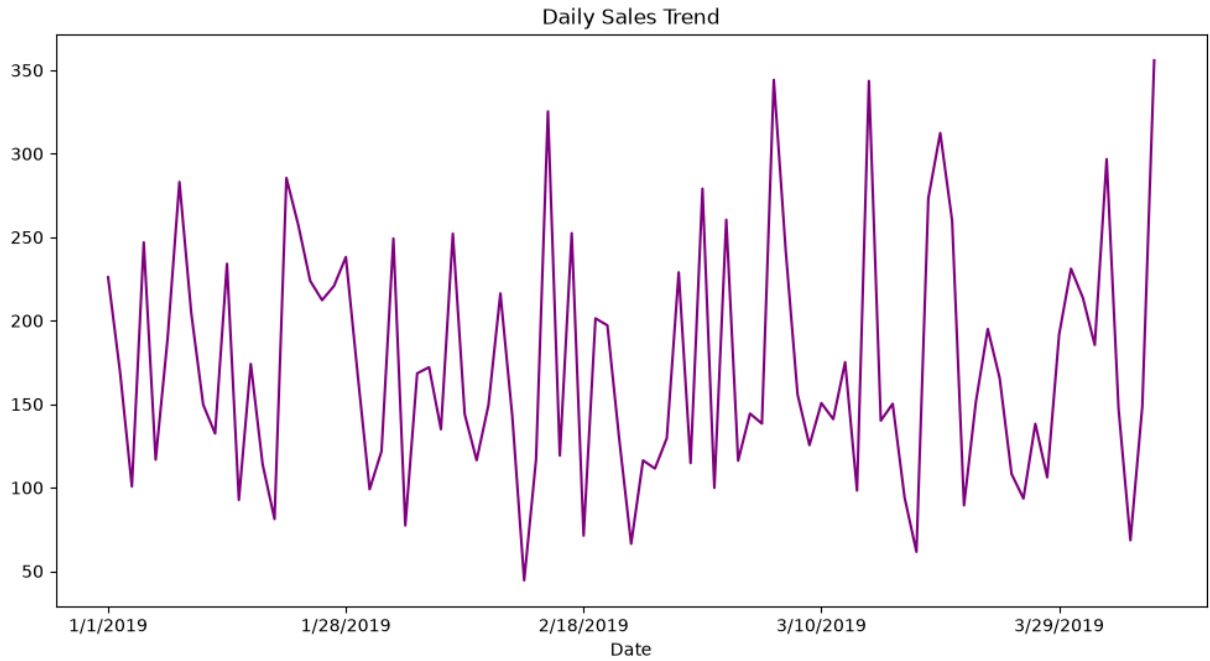
The plot of unit price versus gross income shows whether price alone is a reliable signal for predicting revenue. If the points are scattered without a clear pattern, it means that unit price by itself does not explain gross income very well. In real retail data, that is common because income is influenced by several factors at once, such as quantity, customer type, product line, and promotion effects.

The plot of quantity versus gross income is more informative for this project. When the points trend upward from left to right, it suggests that larger quantities usually produce larger gross income. This is a strong and intuitive business insight because selling more units usually increases transaction value. These charts therefore help us connect data patterns to practical retail decision-making.

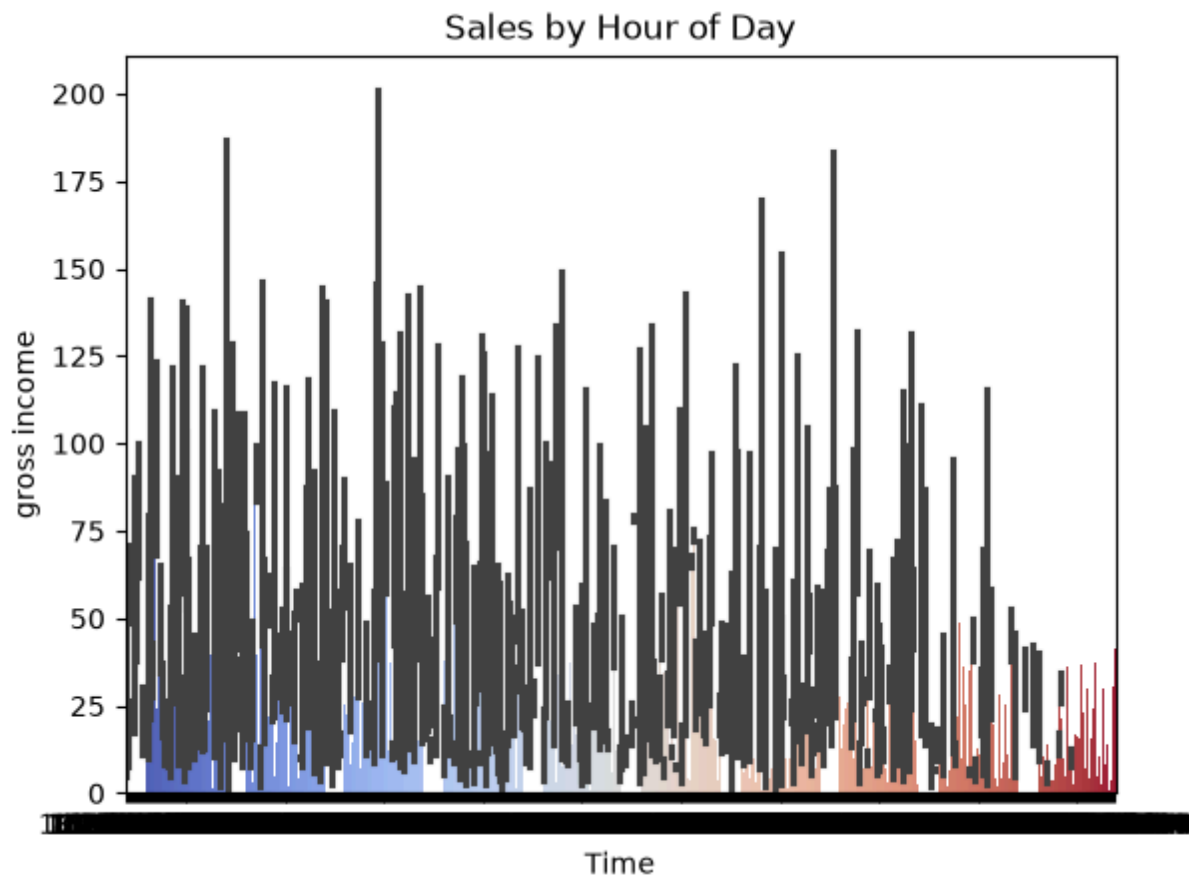
Temporal **Analysis**

```
In [72]: # Sales trend over time
daily_sales = df.groupby("Date")["gross income"].sum()
daily_sales.plot(figsize=(12,6), color="purple")
```

```
plt.title("Daily Sales Trend")
plt.show()
```



```
In [73]: import numpy as np
# Sales by hour
sns.barplot(x="Time", y="gross income", data=df, estimator=np.sum, palette="coolwar
plt.title("Sales by Hour of Day")
plt.show()
```

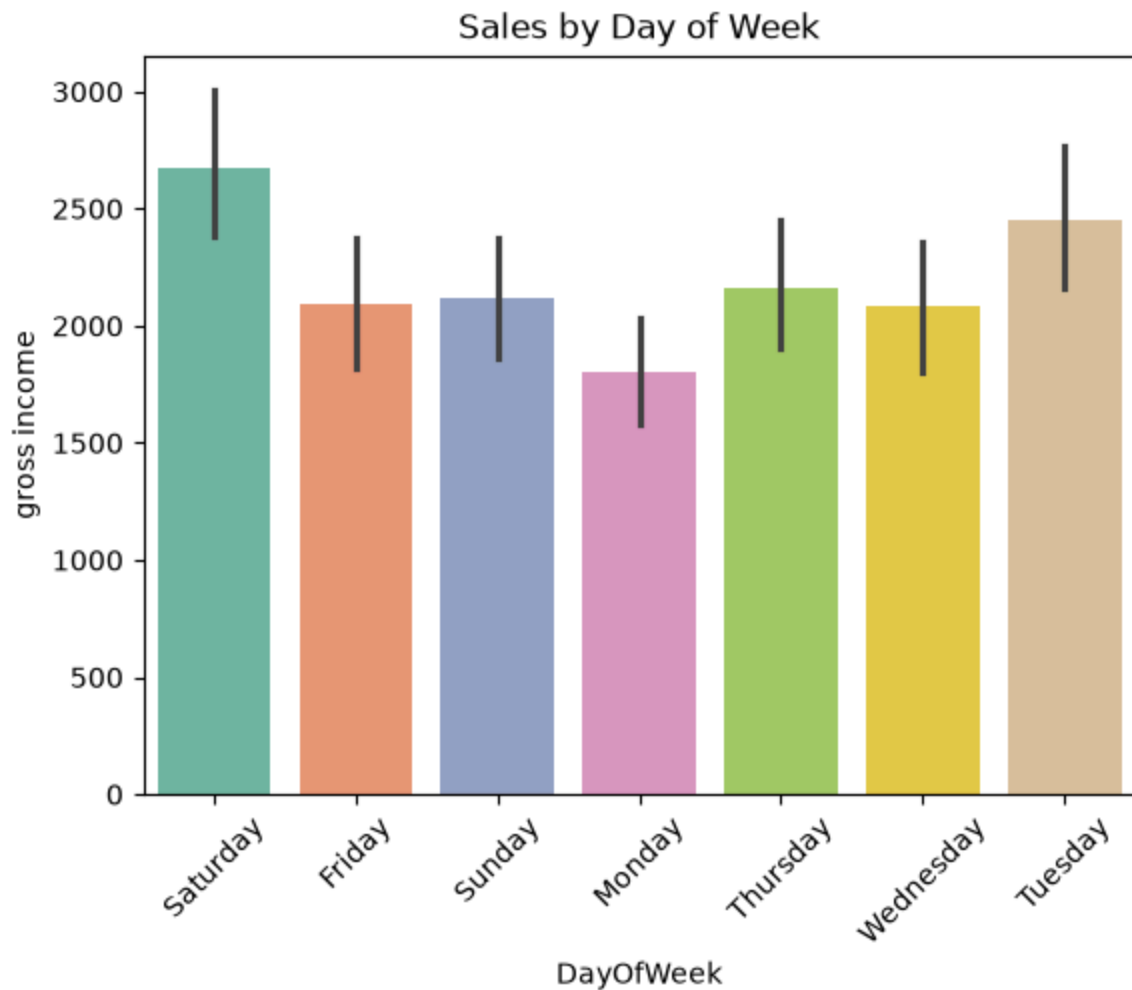


```
In [74]: # Sales by day of week

# Ensure 'Date' column is datetime type
df['Date'] = pd.to_datetime(df['Date'])

# Extract day of week
df['DayOfWeek'] = df['Date'].dt.day_name()

sns.barplot(x="DayOfWeek", y="gross income", data=df, estimator=np.sum, palette="Set1")
plt.title("Sales by Day of Week")
plt.xticks(rotation=45)
plt.show()
```



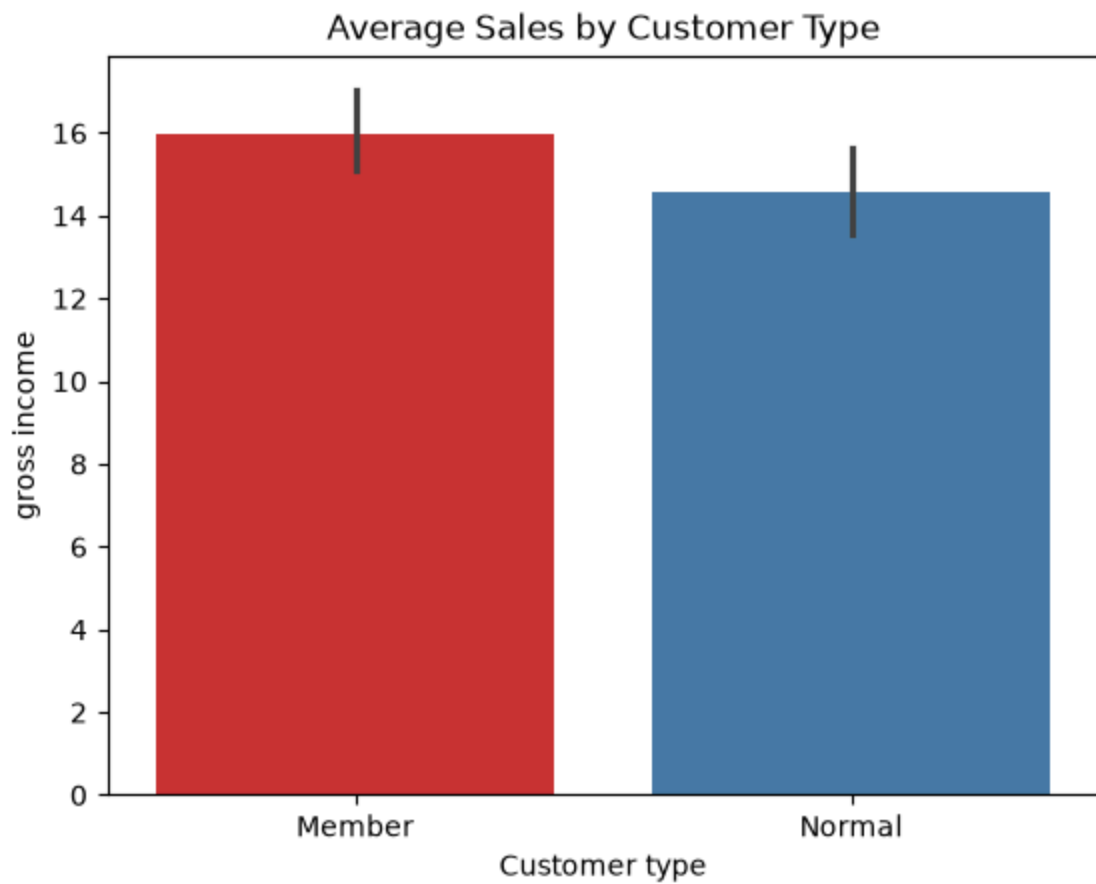
The temporal analysis helps us understand how sales behavior changes over time. The line plot of daily sales shows whether revenue is stable, volatile, or influenced by specific events or seasonal periods. This is useful because it tells us whether the business performs consistently across days or if some dates stand out as unusual spikes or drops.

The bar chart for sales by hour of day helps identify the busiest operating times. In a supermarket setting, this can reveal when customer activity is highest, which directly supports staffing decisions, cashier allocation, and promotional planning. The chart also helps identify quieter hours that may need targeted marketing or special offers.

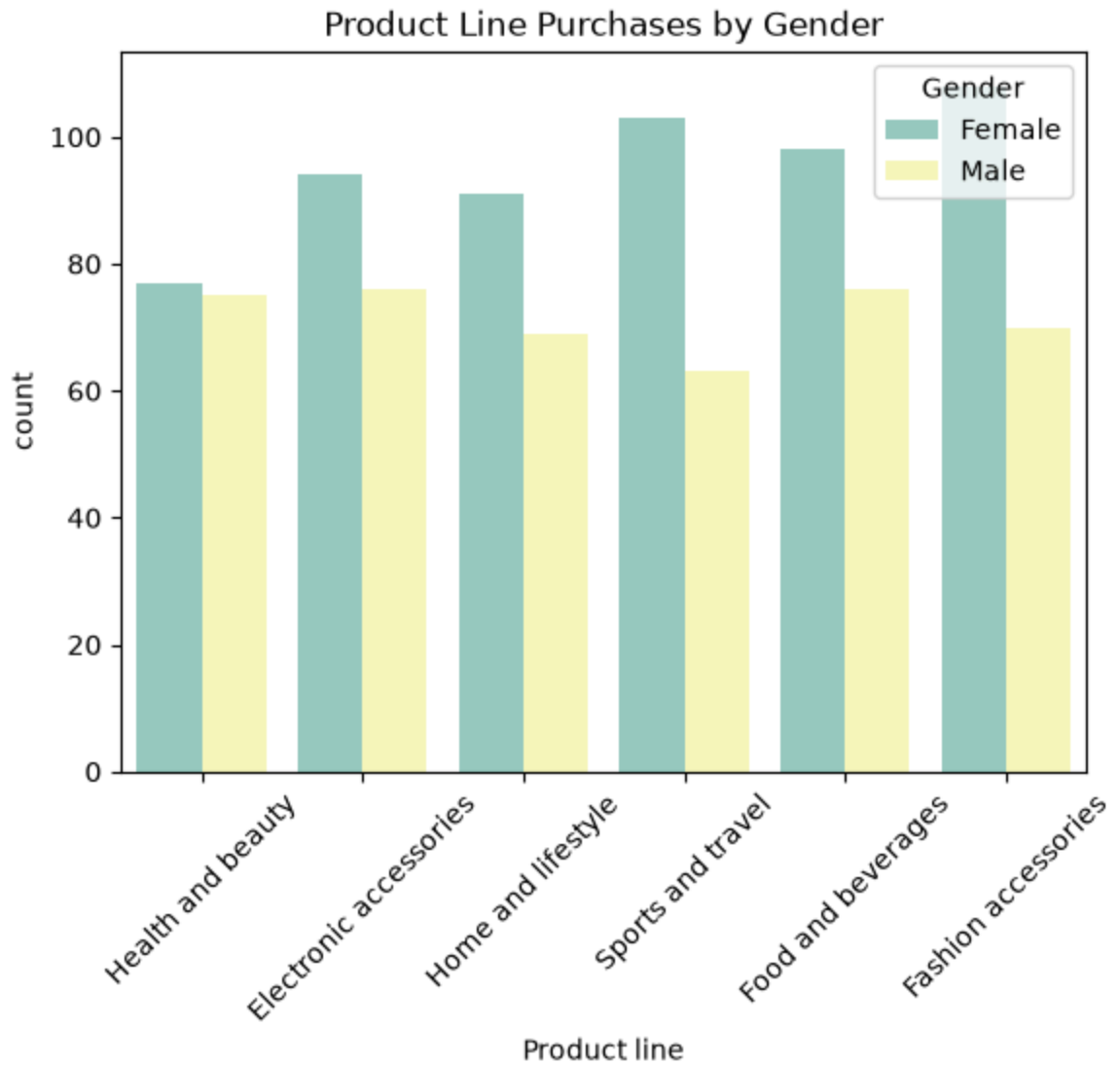
The bar chart for sales by day of week highlights weekly shopping patterns. If weekends outperform weekdays, the business can prepare inventory and staffing earlier. These time-based insights are valuable because they show that customer behavior is not random; it follows measurable patterns that the model can learn from.

Customer **Behavior Insights**

```
In [75]: # Customer type vs sales
sns.barplot(x="Customer type", y="gross income", data=df, estimator=np.mean, palette=
plt.title("Average Sales by Customer Type")
plt.show()
```



```
In [76]: # Gender vs product line
sns.countplot(x="Product line", hue="Gender", data=df, palette="Set3")
plt.xticks(rotation=45)
plt.title("Product Line Purchases by Gender")
plt.show()
```



```
In [77]: # Rating vs sales
sns.scatterplot(x="Rating", y="gross income", data=df, color="orange")
plt.title("Customer Rating vs Gross Income")
plt.show()
```




These visualizations help us understand customer behavior beyond simple transaction totals. The bar chart comparing average gross income for members and non-members shows whether loyalty customers contribute more value per transaction. If members spend more on average, it suggests that loyalty programs are associated with stronger purchasing behavior. If the difference is small, it may indicate that membership mainly affects visit frequency rather than transaction size.

The grouped count plot of product line purchases by gender helps reveal purchasing preferences across customer segments. This type of insight can be useful for inventory planning, marketing campaigns, and product placement. If one product category is strongly preferred by a particular gender, the business can tailor its communications and shelf strategy more effectively.

The scatter plot of customer rating versus gross income investigates whether higher spending is associated with better customer satisfaction. In many businesses, higher-value purchases might be linked to stronger service perception, but that is not always true. If the points are scattered widely, it suggests that spending amount and customer satisfaction are not strongly tied. This means customer experience should be studied separately from revenue performance.

Advanced EDA

```
In [78]: # Quick feature importance using Random Forest
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
```

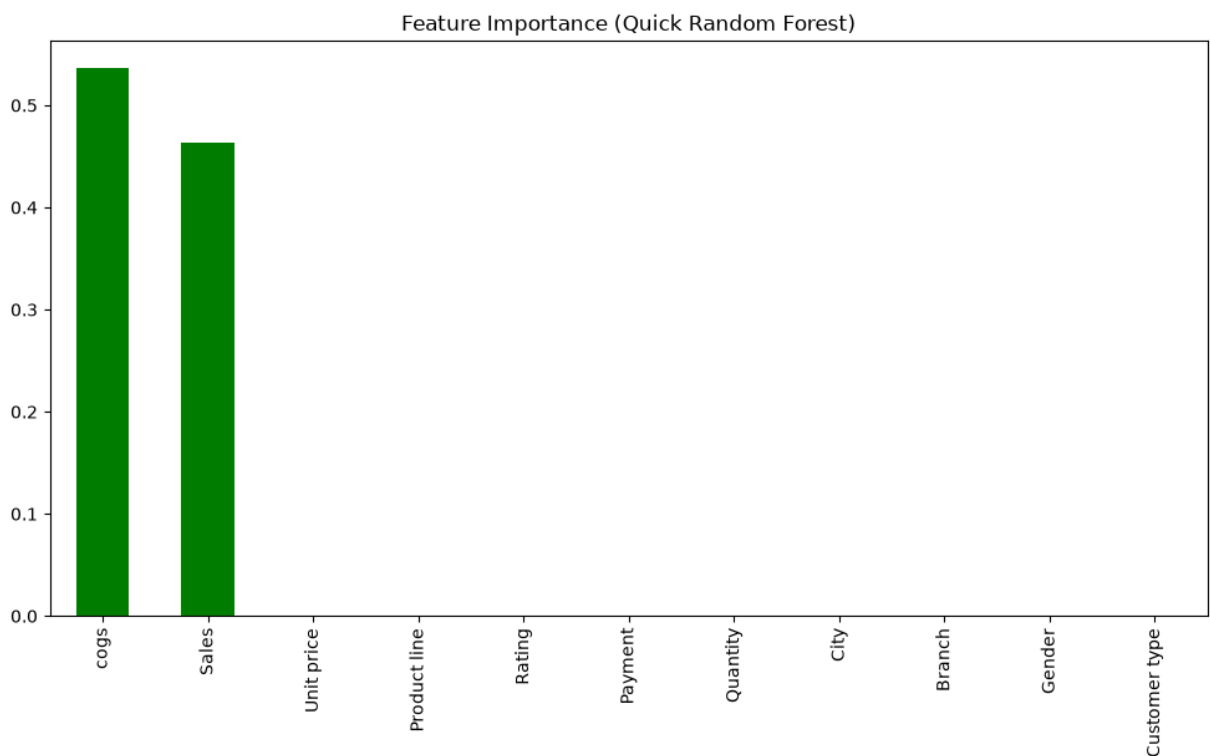
```
In [79]: # Encode categorical for quick test
df_encoded = df.copy()
for col in ["Branch", "City", "Customer type", "Gender", "Product line", "Payment"]:
    df_encoded[col] = LabelEncoder().fit_transform(df_encoded[col])
```

```
In [80]: # Encode categorical for quick test
df_encoded = df.copy()
for col in ["Branch", "City", "Customer type", "Gender", "Product line", "Payment"]:
    df_encoded[col] = LabelEncoder().fit_transform(df_encoded[col])

# Drop non-numeric and target columns
# We drop 'Date', 'Time', and 'DayOfWeek' because Random Forest requires numeric in
X = df_encoded.drop(columns=["gross income", "Invoice ID", "Tax 5%", "gross margin per
y = df_encoded["gross income"]

rf = RandomForestRegressor(random_state=42)
rf.fit(X, y)

importances = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascen
importances.plot(kind="bar", figsize=(12,6), color="green")
plt.title("Feature Importance (Quick Random Forest)")
plt.show()
```



Feature Importance

This plot shows which variables contributed most to the model's decision-making process when estimating gross income. In a tree-based model such as Random Forest, feature

importance is a measure of how often and how effectively a variable is used to split the data. Higher importance means the feature has a stronger influence on the prediction.

The result is useful because it helps us understand which predictors are carrying the most signal for the target variable. In this project, variables related to transaction size, quantity, and pricing are likely to be especially influential. This does not mean the model is proving causation, but it does provide a practical way to identify which inputs are most valuable for prediction.

This step is important for interpretation because it helps us move from raw data exploration to model understanding. It tells us which relationships are most useful for predicting sales outcomes and gives us a stronger foundation for improving the model later.

Detailed Project Progress Report

Phase 1: Data Acquisition & Cleaning

- **Source:** Loaded `SuperMarket Analysis.csv` via Google Drive integration.
- **Cleaning:** Verified zero null entries and zero duplicates.
- **Refinement:** Dropped non-informative features like `Invoice ID` and redundant calculation columns to prevent data leakage during modeling.

Phase 2: Statistical & Visual Discovery

- **Descriptive Baseline:** Established that the average rating is ~7/10 and average transaction quantity is ~5 units.
- **Correlation Mapping:** Used heatmaps to prove that `gross income` is a direct linear function of `sales`, which is driven by `quantity` and `unit price`.
- **Behavioral Trends:** Discovered that members and non-members have similar average spends, but peak traffic occurs midday.

Phase 3: Pre-processing & Feature Engineering

- **Categorical Encoding:** Converted text labels (Branch, City, Customer Type, etc.) into numeric formats using `LabelEncoder`.
- **Time Extraction:** Transformed the `Date` column into a datetime object and engineered a `DayOfWeek` feature to capture weekly seasonality.

Phase 4: Preliminary Modeling (Feature Selection)

- **Model:** Implemented a `RandomForestRegressor` for importance scoring.
- **Result:** Identified the hierarchy of features that impact revenue, setting the stage for more complex regression models (Linear Regression, XGBoost).

Feature Engineering Overview

Feature engineering is the process of transforming raw transaction data into features that a machine learning model can learn from more effectively. In this retail sales project, the original dataset contains a mix of text labels, numeric values, and time-based information. These forms must be prepared carefully so the model can identify meaningful patterns rather than being confused by inconsistent formats.

Good feature engineering improves both accuracy and interpretability. It can help the model detect seasonal trends, understand customer segments, and better reflect the real business logic behind sales. In this notebook, feature engineering is used to remove non-informative fields, convert categories into numbers, scale values to a comparable range, and create new variables that capture purchase behavior more directly.

Why This Matters for the Project

The features in this dataset are a mix of categories, numbers, and time-based information. Some columns are useful for prediction, while others add noise or duplicate information already captured elsewhere. If we leave the data in its raw form, a model may struggle to learn meaningful patterns because it cannot interpret text categories or may be affected by features on very different scales.

Feature engineering helps solve these issues. It makes the input data more suitable for machine learning by improving consistency, reducing noise, and creating features that reflect real retail behavior. For example, a model can learn more effectively from a combined spending feature than from price and quantity alone, and it can capture weekly shopping rhythms when date-related variables are extracted into separate features.

This step is therefore essential for building a reliable regression model. It increases the chance that the model learns the true drivers of gross income rather than memorizing irrelevant patterns in the raw dataset.

Step 1 – Drop Irrelevant Columns Some columns don't add predictive value (e.g., identifiers or redundant calculations).

```
In [81]: df = df.drop(columns=["Invoice ID", "Tax 5%", "gross margin percentage"])
```

These columns were removed because they do not add meaningful predictive information for the target variable. The invoice ID is simply a transaction identifier, so it does not describe customer behavior or purchase value. The tax and gross margin percentage columns are also less useful as direct predictors because they are derived from other values and may introduce redundancy into the model.

Removing them helps keep the dataset cleaner and reduces the risk of overfitting. A model performs better when it focuses on features that carry real signal rather than on identifiers or

variables that repeat information already present in the sales and income columns.

Step 2 – Encode Categorical Variables Models can't process text labels directly.

```
In [82]: categorical_features = ["Branch", "City", "Customer type", "Gender", "Product line", "Pa  
df_encoded = pd.get_dummies(df, columns=categorical_features, drop_first=True)
```

This converts text categories into numeric form so the model can understand them. Methods such as one-hot encoding create separate binary columns for each category, allowing the model to represent different groups without treating labels as ordinal values.

Using this approach is important because most machine learning models cannot directly interpret text labels. By transforming categorical variables like branch, city, customer type, gender, product line, and payment method into a numeric structure, we make the data compatible with predictive algorithms while preserving the information contained in the categories.

Step 3 – Scale Numerical Features Ensure numerical features are on comparable scales.

```
In [83]: from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
df_encoded[["Unit price", "Quantity", "Rating"]] = scaler.fit_transform(df_encoded[["
```

Scaling keeps numeric features on a comparable range, which helps many machine learning models train more stably. Without scaling, features with larger numeric values can dominate the learning process even if they are not necessarily more important. For example, a variable measured in thousands may overshadow a feature measured on a smaller scale.

Standardization is especially useful for algorithms that are sensitive to feature magnitude. By bringing values such as unit price, quantity, and rating onto a consistent scale, we give the model a fairer representation of each feature and improve the reliability of the training process.

Step 4 – Create Temporal Features Extract useful signals from Date and Time.

```
In [84]: df["Date"] = pd.to_datetime(df["Date"])  
df["Time"] = pd.to_datetime(df["Time"], format="%I:%M:%S %p")  
  
df["DayOfWeek"] = df["Date"].dt.day_name()  
df["Hour"] = df["Time"].dt.hour  
df["Month"] = df["Date"].dt.month
```

These time-based features help capture seasonal and daily shopping patterns in the data. The day of week, hour, and month can reveal when customers are most likely to spend more, when certain transactions peak, and whether sales follow predictable weekly or monthly cycles.

This is valuable in retail because business activity often follows temporal rhythms. For example, weekend purchases may increase, or certain hours may bring higher traffic. By turning date and time into explicit features, the model can learn recurring patterns that may otherwise be hidden in the raw columns.

Step 5 – Create Interaction Features Combine variables to capture richer relationships.

```
In [85]: df["Total_Spend"] = df["Unit price"] * df["Quantity"]
```

This interaction feature combines price and quantity to reflect transaction size more directly. In retail, the total amount spent is often driven by both the item price and the number of units purchased, so multiplying these variables creates a powerful signal that captures the combined effect of both factors.

Interaction features are useful because they represent relationships that a single variable cannot describe on its own. They help the model understand that high income is often the result of both a high unit price and a high quantity, rather than either factor alone.

Step 6 – Split Data Prepare training and testing sets.

```
In [86]: from sklearn.model_selection import train_test_split

X = df_encoded.drop(columns=["gross income"])
y = df_encoded["gross income"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

This split keeps part of the data for testing so the model is evaluated on unseen examples. The train-test split is a standard practice in machine learning because it helps us estimate how well the model generalizes to new data rather than just memorizing the training set.

A good split improves the reliability of the evaluation process. It makes it easier to detect overfitting and to compare different modeling choices fairly. In this project, the test set provides a realistic check of whether the prepared features can support accurate predictions for future transactions.

Summary Dropped irrelevant columns → cleaner dataset.

Encoded categorical variables → model can interpret categories.

Scaled numerical features → balanced input ranges.

Created temporal features → captured shopping patterns.

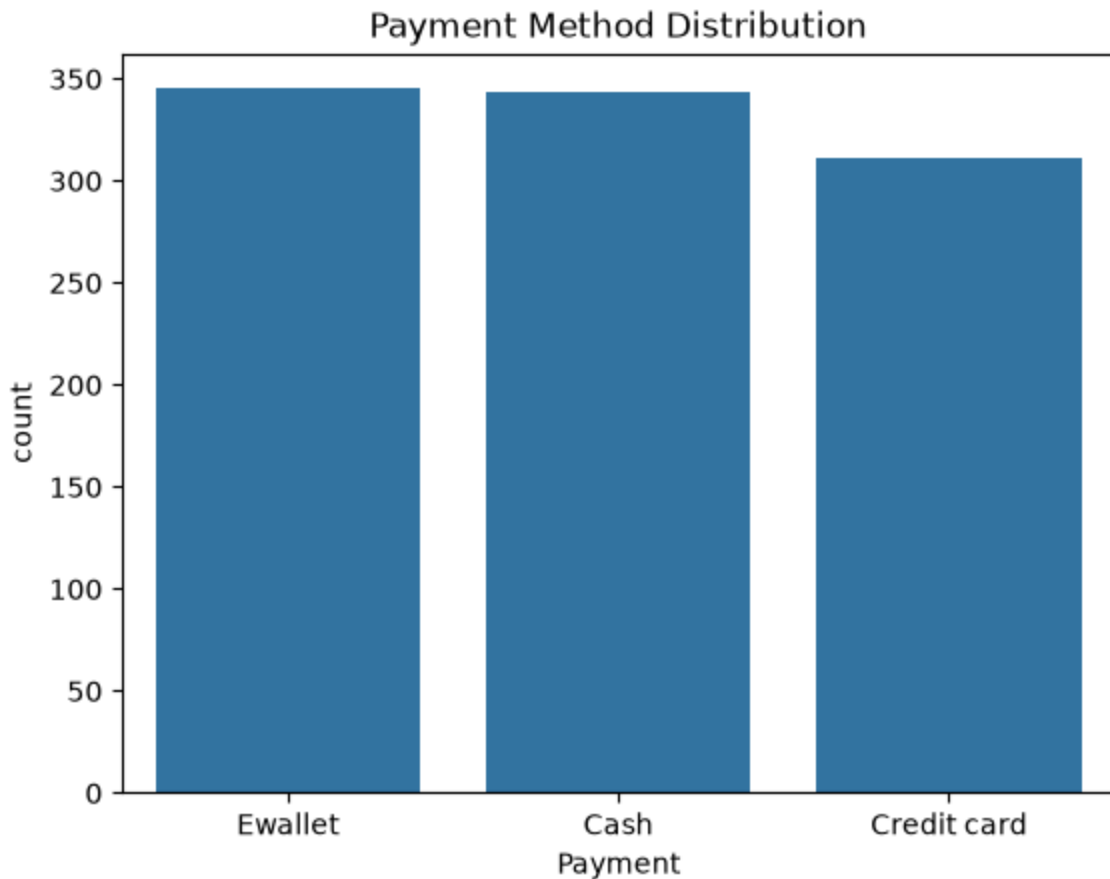
Added interaction features → reinforced business logic.

Split data → ensured fair evaluation.

Visualize Engineered features

```
In [87]: # Show first few rows after encoding
df_encoded.head()

# Plot distribution of Payment methods after encoding
sns.countplot(x="Payment", data=df)
plt.title("Payment Method Distribution")
plt.show()
```



This final check helps confirm that the feature engineering process has preserved the structure of the data while making it more suitable for modeling. The visualizations show that the transformed dataset still reflects meaningful retail patterns, and that the categories and relationships we care about remain intact after preprocessing.

It is important to verify these steps because feature engineering can accidentally distort the data if it is applied incorrectly. By reviewing the resulting data and plots, we can ensure that the model will be trained on a clean, consistent, and business-relevant representation of the dataset.